Practical – 8

Develop a program using malloc(), calloc(), realloc(), and free().Monitor memory allocation behaviour and identify memory leaks using Valgrind.

```
  GNU nano 8.7.1                    memory_alloc.c
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int *a, *b;

    a = (int*)malloc(5 * sizeof(int));

    b = (int*)calloc(5, sizeof(int));

    a = (int*)realloc(a, 10 * sizeof(int));

    free(a);
    free(b);

    return 0;
}
```

```
ketana_sindhu@KetaZone:~$ nano memory_alloc.c
ketana_sindhu@KetaZone:~$ gcc memory_alloc.c -o memory_alloc
ketana_sindhu@KetaZone:~$ ./memory_alloc
ketana_sindhu@KetaZone:~$ valgrind --leak-check=full ./memory_alloc
==2351== Memcheck, a memory error detector
==2351== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2351== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2351== Command: ./memory_alloc
==2351==
==2351==
==2351== HEAP SUMMARY:
==2351==     in use at exit: 0 bytes in 0 blocks
==2351==   total heap usage: 3 allocs, 3 frees, 80 bytes allocated
==2351==
==2351== All heap blocks were freed -- no leaks are possible
==2351==
==2351== For lists of detected and suppressed errors, rerun with: -s
==2351== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
ketana_sindhu@KetaZone:~$
```